32

BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

'Validates' vs 'Checks Whether': Ana Akışta Başarıyı Mühürleme Sanatı

The 'Validates' vs 'Checks Whether' Dictum: Asserting Success in Main Flow

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                'VALIDATES' VS 'CHECKS WHETHER' DİLBİLGİSEL DİKTASI

❌ 'CHECKS WHETHER' (KONTROL EDER - UCU AÇIK & TEHLİKELİ):
"Sistem parolanın doğru olup olmadığını KONTROL EDER."
- Sonuç açıkta kaldı! Doğruysa ne olacak? Yanlışsa ne olacak?
- Kodda iç içe geçmiş devasa if-else spagettisi doğurur!

✅ 'VALIDATES THAT' (DOĞRULAR - BAŞARIYI MÜHÜRLER):
"Sistem parolanın doğru olduğunu DOĞRULAR (BR-001)."
- Ana akışta bu adım KESİNLİKLE BAŞARILIDIR! (Asserts Success!)
- Peki ya parola yanlışsa? Cevap net: Alternatif Akış A1'e git!

🎯 MİMARİ SONUÇ: Ana akış dümdüz bir otoban gibi akar; tüm pürüzler
                 adlandırılmış alternatif akışlara (A1, A2) delege edilir!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Havalimanında pasaport polisine gittiğinde spiker şöyle demez: 'Polis pasaportun sahte olup olmadığına bakıyor ve düşünüyor.' Hayır! Ana senaryoda 'Polis pasaportun geçerli olduğunu doğrular' denir ve yürür geçersin. Eğer pasaport sahteyse, o zaman polis düğmeye basar ve seni yan odaya (Alternatif Akış A1) çekerler! 'Doğrular' kelimesi ana yolda yürümeyi sağlar; soru işaretlerini çöpe atar!

🇬🇧 ENGLISH (ELI5)

At passport control, the script does not read: 'Officer ponders whether the visa is forged.' The main scenario asserts: 'Border officer validates that the visa is authentic.' You proceed immediately to the gate. If forged, that failure diverts cleanly into Alternative Flow A1 (Detention Room). Using 'validates' asserts victory in the happy path and pushes failure into an explicit branch.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

'Validates' Kelimesinin Eklemeli (Additive) Mimari Gücü: Cockburn ve Martinelli'nin en radikal kuralı: 'Validates, never checks whether.' (Doğrular deyin, kontrol eder demeyin!). 'Checks whether' ifadesi iki ucu açık bir olasılıktır ve if-else şartlı dallanmalarını tetikler. Use case'in ana senaryosuna if-else sokulamaz! 'Sistem üyenin gecikmiş borcu olmadığını doğrular (BR-006)' dediğinizde, ana akışta bu adımın %100 başarılı olduğu mühürlenmiş olur. Bu tek bir kelime tercihi sisteme şu muhteşem mimari faydaları sağlar: 1) Eklemeli Yapı (Additive Structure): Ana senaryo baştan sona pürüzsüz akar; her bir başarısızlık ihtimali ise aşağıda bağımsız bir alternatif akış (A1, A2) olarak tanımlanır. 2) AI'a Hata Şablonu Verme: AI ajanı bu yapıyı gördüğünde kodun içinde spagetti if-else blokları kurmaz; doğrudan `if (!valid) throw new ValidationException()` veya Result/Either monadı üreterek akışı ilgili alternatif metoda delege eder.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Additive Architecture of 'Validates' vs 'Checks Whether': 'Checks whether' is an open-ended predicate that invites inline branching and nested conditionals within the main scenario. The main scenario must remain strictly linear and unconditionally successful. Writing 'System validates that password matches hash (BR-001)' asserts success in the primary flow. This single verb choice constructs the additive architecture of the specification: 1) The main scenario is an unbranching highway where every validation passes. 2) Deviations are strictly quarantined into numbered alternative flows. 3) For generative AI, this enforces clean architectural patterns: guard-clause rejections (`if (!valid) throw new DomainException()`) rather than tangled if-else pyramids.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 'Validates' Kuralının Guard-Clause Mimarisiyle Temiz İcrası
package com.maang.aiup.validates;

public class ValidationGuardArchitecture {
    // ❌ CHECKS WHETHER (Spagetti if-else):
    // if (checkPassword()) { if (checkLimit()) { doSomething(); } else { error(); } }

    // ✅ VALIDATES (Guard Clauses - Şartnamedeki her 'validates' bir guard'dır):
    public static void processBorrowWithGuards(boolean isCopyAvailable, boolean isLimitOk) {
        // Step 4: System validates copy is available (A1)
        if (!isCopyAvailable) {
            handleAlternativeA1();
            return;
        }

        // Step 5: System validates loan limit (A2)
        if (!isLimitOk) {
            handleAlternativeA2();
            return;
        }

        // Ana akış dümdüz devam eder!
        System.out.println("Tüm doğrulamalar geçti, ana akış başarıyla bitti.");
    }

    private static void handleAlternativeA1() { System.out.println("Dallanma A1: Kopya yok."); }
    private static void handleAlternativeA2() { System.out.println("Dallanma A2: Limit aşıldı."); }

    void main() {
        processBorrowWithGuards(true, true);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnamede 'Sistem kullanıcının bakiyesini kontrol eder' yazıp bırakırsanız AI bakiye yetersiz olduğunda ne yapacağını bilemez ve kafasına göre kullanıcıya eksi bakiye tanımlayabilir! 'Sistem bakiyenin yeterli olduğunu doğrular (BR-002)' yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Writing 'System checks customer balance' without asserting success leaves the failure path undefined. An AI agent might hallucinate overdraft credit. Write 'System validates that balance is sufficient (BR-002)' and bind failure to flow A1.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- 'Validates' ana senaryoda kesin başarıyı mühürler; 'checks whether' ise ucu açık if-else yaratır.
- Her 'validates' adımı başarısızlık ihtimalini isimlendirilmiş bir Alternatif Akışa devreder.
- Bu kural sayesinde ana akış dümdüz bir başarı otobanı gibi tertemiz kalır.

🎯 Key Architectural Takeaways

- 'Validates' asserts success in the main flow; 'checks whether' creates ambiguous branching.
- Every validation step delegates its failure outcome to a named alternative flow.
- This rule preserves the happy path as a linear, unbranching highway to the goal.